Scenario-Based Interface Problems

Level: Advanced / Industry-Oriented
Focus: Interface-driven design, loose coupling, runtime polymorphism, default and static methods in interfaces.

Instructions:
- Use interfaces strictly
- Avoid if-else or switch-based logic in main flow
- Focus on clean design and extensibility

Problem 1: Enterprise Notification Dispatch System

A large enterprise application sends notifications via multiple channels such as Email, SMS, Push Notification, and Slack.

Design an interface NotificationService with method:
sendNotification(String message, String recipient)

Implement EmailNotification, SMSNotification, and PushNotification classes.

Requirements:
- Notification type should be decided at runtime
- Main application must use interface reference only
- Add a default method for logging notification activity

Problem 2: Pluggable Authentication Framework

A corporate system supports multiple authentication mechanisms such as Password-based login, OTP-based login, and Biometric authentication.

Design an interface AuthenticationProvider with method:
boolean authenticate(String userId, String credential)

Implement PasswordAuthProvider, OtpAuthProvider, and BiometricAuthProvider.

Requirements:
- Authentication strategy must be interchangeable
- Add a static method in interface to validate userId format
- Main application must remain unchanged for new auth types

Problem 3: Multi-Vendor Cloud Storage System

A SaaS application allows users to store files across multiple cloud providers such as AWS, Azure, and Google Cloud.

Design an interface CloudStorageService with methods:
uploadFile(String fileName)
downloadFile(String fileName)

Implement AwsStorageService, AzureStorageService, and GcpStorageService.

Requirements:
- Cloud provider must be swappable without code changes
- Add default retry logic in interface
- Track upload count using static variable in interface

Problem 4: Online Order Pricing Engine

An e-commerce platform applies different pricing strategies based on customer type and promotional events.

Design an interface PricingStrategy with method:
double calculatePrice(double basePrice)

Implement RegularPricing, FestivalPricing, and PremiumPricing strategies.

Requirements:
- Pricing logic must be selected at runtime
- Avoid conditional logic in main application
- Add default tax calculation method in interface

Problem 5: Enterprise Report Generation Engine

A reporting system generates reports in multiple formats such as PDF, Excel, and JSON.

Design an interface ReportGenerator with method:
void generateReport(Object data)

Implement PdfReportGenerator, ExcelReportGenerator, and JsonReportGenerator.

Requirements:
- Report format must be decided at runtime
- Add default validation method in interface
- Add static helper method for timestamp generation